

# **Cesar Moreno Fernandes - Reliability Lessons From SQLite — Richard Hipp**

## **Bloco 1 – Cultura de Testes e Engenharia de Software**

### **1. Código x Código de Teste**

A visão tradicional é a de que escrever uma quantidade muito grande de testes pode ser desperdício de tempo, principalmente porque os testes aumentam o tamanho do projeto e podem exigir mais trabalho dos desenvolvedores. O SQLite mostra que essa visão pode estar errada, pelo fato dos testes serem uma das principais razões pelas quais o SQLite consegue manter um nível tão alto de confiabilidade com uma equipe muito pequena. A filosofia do projeto é testar praticamente todas as possibilidades importantes e detectar problemas antes que eles cheguem aos usuários.

### **2. Impacto na Performance**

Os testes permitem que os desenvolvedores façam alterações profundas no código e otimizações de desempenho sabendo que podem verificar rapidamente se o comportamento anterior continua correto. Assim, uma implementação mais rápida pode substituir uma implementação antiga sem que o programador precise confiar apenas em testes manuais. Sendo assim, os testes não necessariamente têm um impacto negativo na performance, considerando que desenvolvedores os usem como auxílio para otimizar o código.

## **Bloco 2 – Técnicas de Teste na Prática**

### **3. Simulação de Falhas (Fault Injection)**

O SQLite utiliza a fault injection, traduzindo, injeção de falhas, para simular situações que seriam muito difíceis de reproduzir normalmente. Ela permite que o sistema simule, por exemplo, uma falha de memória. O mesmo princípio é aplicado a problemas relacionados à persistência e segurança dos dados. A equipe consegue simular interrupções e situações anormais durante operações de escrita, verificando se o banco consegue recuperar seu estado corretamente.

### **4. Fuzzing e Testes de IA**

Antes de responder a pergunta, é importante notar que cada método funciona de forma diferente e serve para detectar situações e erros diferentes. MCDC (Modified Condition/Decision Coverage) testa cada condição no código, ou seja, se um if tem várias condições, serão feitos vários testes para ver se cada uma consegue influenciar o resultado individualmente. O Fuzzing realiza uma quantidade enorme de entradas

aleatoriamente, com o intuito de ver se acontece alguma coisa inesperada ou uma falha no código. O Fuzzing Profile-Guided é uma evolução disso, analisando quais partes do código foram mais utilizadas para poder também focar em partes menos utilizadas. A IA olha o código diretamente, em vez de testar entradas, analisando a lógica e identificando possíveis erros, além de criar testes específicos. A IA encontrou que, em um caso específico, o quicksort rearranjava incorretamente os valores. Isso foi possível pois ela funciona de forma drasticamente distinta de outras formas de testes, assim conseguindo acessar casos de erros muito específicos que nem o MCDC e o Fuzzing conseguem.

## **Bloco 3 – Reflexão Crítica**

### **5. A "Profissão Tester"**

A fala mostra que não há nenhuma vergonha em ser “apenas um testador”. Os testes são uma das partes mais importantes na integridade de um projeto. Um Engenheiro de Qualidade ajuda a construir sistemas mais confiáveis, identificando erros e criando estratégias para encontrar falhas. E não é apenas uma pessoa ou equipe de testes que é responsável pela qualidade, mas sim toda a equipe de desenvolvimento, durante todo o processo. O SQLite mostra isso claramente. A enorme quantidade de testes e a automação permitem que uma equipe extremamente pequena mantenha um software utilizado em uma escala gigantesca.

### **6. Conclusão – Frases e conceitos marcantes**

A ideia de que o código de testes pode ser muito maior que o código do programa foi a que mais me marcou. É muito comum ter a ideia de que um código mais compacto e simples é sempre melhor que um código complexo e extenso, e por bom motivo, já que isso é algo que nos deixa mais atento a códigos redundantes que complicam o projeto, que estão muito presentes principalmente quando desenvolvedores usam a IA sem sequer verificar o que está sendo gerado. Eu mesmo tive essa ideia por muito tempo, dito isso, Richard mostra que há um pouco mais de nuance nela, sendo que ele fez a escolha deliberada e calculada de incluir uma quantidade enorme de código que só serve para testes. A diferença disso e apenas um código redundante é que os testes tiveram muita importância na otimização e segurança do programa, consolidando-o como um dos mais utilizados softwares de banco de dados atualmente.